

Forward Deployed Engineering: A Systems Engineering Perspective

Advait Dharmadhikari

Abstract

Technology vendors increasingly place their own engineers inside customer organizations to make artificial intelligence systems work in production, a practice the industry calls forward deployed engineering. The practice is expanding rapidly and is discussed almost entirely outside the peer-reviewed literature. This paper reports a structured multivocal review of 133 academic and practitioner sources. We find that the peer-reviewed literature contains no study of the role, that the quantitative claims circulating about it fail provenance checks, and that the two research traditions capable of explaining it, software engineering for artificial intelligence and enterprise implementation research, each hold one half of the problem while neither holds both. We then analyze forward deployed engineering as a systems engineering problem, treating the vendor and client together as a system of systems with independent managerial control, and map the reported activity onto the ISO/IEC/IEEE 15288 process model. Three structural properties follow: requirements are discoverable only through operation, verification evidence must be produced inside an environment the supplier does not control, and delivered architecture mirrors the delivery organization across the firm boundary. We derive a cross-organizational deployment lifecycle, locate the capability-deployment verification gap within it, and set out a research agenda. We argue that forward deployed engineering is best theorized not as a new profession but as a transitional organizational form whose eventual dissolution would mark its success.

Keywords: *forward deployed engineering; AI engineering; systems engineering; enterprise AI deployment; multivocal literature review; ISO/IEC/IEEE 15288*

1. Introduction

A distinctive organizational form has appeared around the commercialization of artificial intelligence. Rather than shipping a product and supporting it at arm's length, vendors are placing their own engineers inside customer organizations for extended periods, where those engineers integrate the product with client systems, build customer-specific components on top of it, and carry what they learn back into the vendor's roadmap. The industry calls these people forward deployed engineers. The label originated at Palantir Technologies in the early 2010s and has since been adopted by frontier model laboratories, data platform vendors and financial technology firms.

The practice is growing quickly enough to have become a recruiting phenomenon, and it is discussed almost entirely in company engineering blogs, practitioner newsletters,

consultancy reports and trade journalism. That would be unremarkable if the underlying problem were also unremarkable. It is not. The literature on deploying machine learning systems has documented for a decade that the difficulty of putting a model into production lies overwhelmingly outside the model. Sculley et al. (2015) named the hidden technical debt that accumulates around a learned component. Paleyes et al. (2022) catalogued deployment obstacles across the workflow from published industrial cases. Sambasivan et al. (2021) found compounding upstream data problems in ninety-two per cent of the high-stakes projects they studied, typically surfacing at deployment. Beede et al. (2020) showed a laboratory-accurate diagnostic model degrading in eleven clinics because of lighting, connectivity and patient flow. Shankar et al. (2024) reported from eighteen practitioners that production machine learning remains manual and iterative despite a decade of tooling.

What that literature does not do is treat the response to this difficulty as an object of study. Deployment problems are consistently framed as technical problems awaiting technical solutions, usually platforms. Yet the response emerging in industry is not primarily a platform. It is a staffing decision: put an engineer who owns the product inside the organization that owns the problem. Nobody has asked what kind of engineering that is, what process model it follows, or what it implies for the disciplines that claim to govern system development.

There is a second silence, and it is older. Solutions engineering, sales engineering and the field application engineer are roles that have existed for decades. Our review located no dedicated peer-reviewed research literature on any of them. The academic study of Palantir, the firm that popularized the forward deployed model, is entirely concerned with surveillance and public-security regulation and says nothing about how the company builds and delivers. A class of engineering work that employs a large population of practitioners has escaped scholarly attention almost completely.

This paper addresses that gap from a systems engineering standpoint. Our position is that forward deployed engineering is not adequately described as a job title, a sales function or a species of consulting, and that its distinctive properties become visible only when the vendor and the client are analyzed together as a single engineered system under divided managerial control. We ask three questions. First, what does the available evidence actually establish about forward deployed engineering, and how reliable is that evidence? Second, how does the reported activity map onto an established systems engineering process model, and what does it reveal when the mapping fails? Third, what follows for research and for practice?

The paper makes five contributions. It provides the first structured multivocal review of forward deployed engineering, spanning peer-reviewed, preprint, practitioner and trade sources under an explicit evidence-tiering scheme. It reports an evidence-provenance audit demonstrating that the headline quantitative claims about enterprise artificial intelligence failure and about the size of the forward deployed population do not survive scrutiny, and it supplies citable alternatives. It offers a refined definition and a comparison that separates forward deployed engineering from six adjacent disciplines with which it is routinely conflated. It derives an activity taxonomy and a cross-organizational deployment lifecycle mapped to ISO/IEC/IEEE 15288, identifying three structural properties that standard process models do not accommodate. Finally, it sets out a research agenda, and argues that the phenomenon is better theorized as a transitional organizational form than as an emerging profession.

We are equally clear about what the paper is not. It is a review and a conceptual contribution: no forward deployed engineer was interviewed and no deployment was observed. Section 3.5 states the consequences and Section 6.1 identifies the studies that would remedy them. Section 2 reviews the relevant literatures, Section 3 sets out the method, Section 4 reports findings, Section 5 develops the systems engineering account, and Sections 6 and 7 discuss and conclude.

2. Background

2.1. The emergence of a practice

The forward deployed model is generally traced to Palantir Technologies, where engineers were embedded with customers to build applications on the company's platform rather than to configure a finished product. Practitioner accounts describe a period in which the firm employed more engineers in customer-facing delivery than in core product development, and an organizational split between analyst teams that identified high-value problems inside the client and engineering teams that built rapid prototypes against them. The model has since diffused. Frontier model laboratories, data platform vendors and application companies now advertise the role, and at least one laboratory has organized a dedicated deployment function around it.

Grey sources converge on a small set of claims about what distinguishes the role. Forward deployed engineers are said to spend a substantial share of their time physically or virtually inside the customer organization; to hold a hybrid skill profile spanning software engineering, systems integration and commercial judgement; and, critically, to influence the vendor's product roadmap rather than merely consuming it. That last claim is the operational test that practitioner accounts use to separate the role from consulting and from the solutions architect function, and it is the property that makes the role theoretically interesting.

One scholarly artefact addresses the term directly. Kim and Hwang (2026) define forward deployed engineering as the embedding of product-company engineers at client sites to deploy, customize and co-evolve software systems, identify product ownership, bidirectional knowledge flow and a

productization feedback loop as its defining properties, and propose a generational taxonomy. Their work is a short working paper without an empirical component, and it positions the practice within boundary spanning and technology transfer. We adopt their three properties as a starting point and extend them, but our analytical route is different: where they locate the practice in organizational theory, we locate it in systems engineering.

2.2. Deployment as an engineering problem

The software engineering literature on artificial intelligence systems has consolidated substantially. Martinez-Fernandez et al. (2022) map the field across 248 studies to early 2020. Amershi et al. (2019) describe a nine-stage workflow and an explicit team maturity ladder from inside a large software organization. Nahar et al. (2022) document collaboration failures across the boundaries between data science, software engineering and operations. Bogner et al. (2021) extend the technical debt account with data, model, configuration and ethics debt.

Operational practice has been codified as MLOps. Kreuzberger et al. (2023) supply the consolidated definition, principles, components, architecture and role taxonomy; Steidl et al. (2023) disambiguate the surrounding terminology across 151 sources; John et al. (2021) contribute a staged maturity model; and Amou Najafabadi et al. (2024) map forty-three studies and conclude that the field possesses architectural components but no agreed reference architecture. For foundation models the equivalent literature is younger and thinner, and we located no field study of that practice comparable in rigour to Shankar et al. (2024) for machine learning.

Two properties of this literature matter for our argument. The first is that it is overwhelmingly intra-organizational. Its implicit unit of analysis is a team inside a firm that owns its data, its infrastructure and its deployment target. The role taxonomy in Kreuzberger et al. (2023) contains no customer-facing role. The second is that where the literature does confront the organizational boundary, it does so briefly and in passing. Granlund et al. (2021) is, to our knowledge, the only paper in the operational machine learning literature that treats multi-organization delivery as a first-class problem, reporting from two industrial cases that data ownership, responsibility division, regulatory constraint and pipeline handoff all become open questions the moment a pipeline crosses a firm boundary. It is four pages long.

2.3. The organizational boundary in enterprise implementation

A mature literature does address engineers working inside client organizations, and it predates artificial intelligence by decades. Tushman (1977) established boundary-spanning roles as informally emergent and tied to task uncertainty, and Ancona and Caldwell (1992) moved the unit of analysis to the team. Ko et al. (2005) validated a measurement model for knowledge transfer from consultants to clients across ninety-six enterprise system implementations. Tuli et al. (2007)

reframed business solutions from product bundles to relational processes spanning requirements definition, customization and integration, deployment and post-deployment support, locating a substantial share of the causes of failure on the client side. Van Beijsterveld and van Groenendaal (2016) distinguished actual from perceived misfits in enterprise resource planning and found organizations customizing to solve problems that did not require it.

This literature supplies the vocabulary that the artificial intelligence deployment literature lacks. It also carries an assumption that forward deployed engineering violates. In all of it, the embedded expert is a third party. Consultants are paid to solve the client's problem and leave. A forward deployed engineer remains a vendor employee whose compensation, career and incentives sit with the supplier, and whose distinctive claim is that customer work improves the vendor's product. That inverts the incentive structure the consulting literature models, and it is the specific configuration that no existing literature addresses.

Adjacent work supplies further leverage. Baxter and Sommerville (2011) propose socio-technical systems engineering as an integrative discipline and explain why socio-technical methods never entered mainstream systems engineering practice. Maier (1998) supplies the criteria, operational and managerial independence in particular, that determine whether a collection of systems should be treated as a system of systems. Conway (1968) proposed that a designing organization reproduces its own communication structure in the system it builds, and McCormack et al. (2012) tested the proposition empirically, finding that loosely coupled organizations produce measurably more modular products.

2.4. Prior art in engineering methodology

Contemporary work on engineering artificial intelligence agents has begun to produce methodological artefacts. Liu et al. (2024) derive eighteen architectural patterns for foundation model based agents from a systematic review, with a decision model for pattern selection. Xia et al. (2024) propose a process model and reference architecture that embed evaluation as a continuous function rather than a terminal checkpoint. Wu (2024) asks directly whether the V-model addresses collaboration problems in building machine learning enabled software, and concludes that its characteristics align with several practitioner difficulties at the cost of additional effort.

None of this work models a specific customer organization. That absence is striking because the problem was addressed twenty years ago. Agent-oriented software engineering produced methodologies that treated a multi-agent system as a computational organization of roles, permissions and protocols (Wooldridge et al., 2000), and Tropos made early requirements analysis of the stakeholder organization a first-class development phase (Bresciani et al., 2004). Understanding the client organization before specifying the system is, in essence, the forward deployed engineering activity, formalized in 2004. The contemporary literature does not cite it. Recovering that prior art is part of what a systems engineering perspective can offer.

3. Method

3.1. Structured multivocal review

Because the phenomenon is documented mainly outside academic publishing, a conventional systematic literature review would return almost nothing. We therefore conducted a multivocal literature review, following the guidelines of Garousi et al. (2019), which extend systematic review procedure to grey literature and specify how to include it without surrendering rigour.

Approximately 120 distinct queries were run across Google Scholar, arXiv, SSRN, the ACM Digital Library, IEEE Xplore, Semantic Scholar and Crossref, and across the engineering publications of named vendor organizations, covering the role (forward deployed engineer, solutions engineer, field application engineer, customer-embedded engineering), the problem (machine learning deployment, production artificial intelligence, last mile of artificial intelligence) and the adjacent disciplines (MLOps, LLMOps, AI engineering, enterprise artificial intelligence architecture). Backward and forward snowballing was applied to all included academic sources.

A source was included if it described the practice of embedding vendor engineers with customers, or a named equivalent; was publicly retrievable and citable with an identifiable author or corporate author and a date; and contained descriptive content about the work, the process or the organization rather than only a product claim. Recruitment advertisements were excluded. The resulting corpus comprises 133 sources.

3.2. Evidence tiering

Grey literature varies enormously in what it can support. We assigned every source to one of four tiers. Tier 0 is peer-reviewed academic literature. Tier 1 is preprints and institutional reports carrying named authors and a stated method. Tier 2 is named-author practitioner accounts and primary company engineering writing. Tier 3 is consultancy output and trade journalism with no disclosed method.

We then applied a single governing rule: no finding reported in this paper rests on a Tier 3 source alone. Tier 3 material may motivate, illustrate or corroborate, and it is cited as such. It never carries a claim. We state the rule explicitly because the principal risk in a multivocal review is that producer interest enters the findings unmarked, and because the Tier 3 material in this domain is produced largely by organizations that sell recruitment, consulting or benchmarking services into the phenomenon they describe.

Every source additionally carries a verification status. Verified means a publisher, repository or institutional record was retrieved and its metadata read directly. Partially verified means title, venue and identifier were confirmed but the full record was inaccessible behind automated-access restrictions. No source at partially verified status carries a substantive claim in this paper.

3.3. Analytical frame

We analyzed the corpus against ISO/IEC/IEEE 15288:2023, the international standard for system life cycle processes. The standard was chosen for three reasons. It is domain-neutral, so it does not presuppose a software or a machine learning framing. It deliberately declines to prescribe a life cycle model, which leaves room for a domain-specific derivation without contradicting the standard. And it distinguishes technical, technical management, agreement and organizational project-enabling process groups, which is precisely the distinction that a cross-organizational analysis requires, because the agreement processes are the ones that become load-bearing when a life cycle spans two firms.

Activities described in the corpus were coded against the standard's process categories, with inductive categories opened where reported activity had no home in the standard. Those inductive categories are reported as findings rather than treated as coding failures, on the argument that a mismatch between documented practice and a mature process model is informative about the practice, the model, or both.

3.4. Design science positioning

The paper produces an artefact, and we position it accordingly, following Hevner et al. (2004). Using the knowledge contribution framework of Gregor and Hevner (2013), the contribution is an exaptation: a known solution, the systems engineering life cycle process model, extended to a new problem domain, cross-organizational artificial intelligence deployment. We claim neither invention nor routine design. The taxonomy in Section 4.4 was developed following the conceptual-to-empirical procedure of Nickerson et al. (2013), with their ending conditions applied to the corpus.

3.5. Limitations

Four limitations bound what follows. First, the review is structured but not preregistered, and saturation was assessed by the absence of new concepts across late search rounds rather than by a formal preregistered criterion. Second, coding was performed by the authors without an independent second coder; the coding frame is published so that the analysis can be reproduced and contested. Third, the corpus skews toward English-language sources and North American technology firms, which almost certainly overstates the homogeneity of the practice. Fourth, and most consequentially, this is a documentary study. Practitioner accounts describe the work as its producers wish it understood, and company engineering writing serves recruitment and positioning as well as description. No forward deployed engineer was interviewed and no engagement was observed. The artefact derived in Section 5 is therefore a theoretically grounded proposal awaiting field validation, and we identify the studies that would provide it in Section 6.1.

4. Findings

4.1. The state of the evidence

The first finding concerns the evidence base itself, and it is a precondition for everything else. The peer-reviewed literature on forward deployed engineering is empty. Searches for the

term and its immediate variants returned no peer-reviewed publication. The same searches for solutions engineering, sales engineering and the field application engineer, treated as research objects rather than as job categories, likewise returned no dedicated research literature, despite these being roles of long standing. Academic work on the firm most associated with the practice concerns surveillance and regulation, not delivery. The one scholarly artefact addressing the term is a short working paper (Kim and Hwang, 2026).

This is not a null result of the ordinary kind. A class of engineering work employing a substantial population of practitioners, at the interface between vendors and customers in a rapidly growing industry, has been almost entirely unstudied, and the absence is independently verifiable by replicating the search protocol.

The second finding concerns the quality of the quantitative claims in circulation. Practitioner and consultancy discourse rests on a small number of figures that are widely repeated and poorly sourced. We traced each to origin; the results are in Table 1.

Table 1. Provenance audit of headline quantitative claims.

Claim	Origin and method	Status
95 per cent of organizations obtain zero return from generative artificial intelligence	Self-published industry report. 52 interviews and 153 survey responses recruited at four industry conferences; self-described as preliminary findings; no sampling frame or instrument disclosed; the derivation of the figure does not appear in the document	Not citable as a rate. Report as a contested industry claim with the sample stated
85 per cent of artificial intelligence projects fail	A 2018 analyst prediction that projects would deliver erroneous outcomes due to bias in data, algorithms or teams. It is a forecast about output quality, not a measured project failure rate; sample undisclosed	Not citable, and routinely misquoted. The misquotation should be corrected wherever it appears
More than 80 per cent of artificial intelligence projects fail	Repeated in a 2024 institutional report which attributes it to a 2022 magazine article. The citation chain terminates in journalism	Not citable as a rate. The report's own 65 practitioner interviews are citable as qualitative evidence
Fewer than 6 per cent of firms used any of five artificial intelligence technologies; just over 18 per cent weighted by employment	National statistical agency business survey of approximately 850,000 firms, published in a peer-reviewed economics journal (McElheran et al., 2024)	Citable. The only peer-reviewed probability-sample adoption baseline located. Caveat: 2018 data, predating large language models
Population of forward deployed engineers: approximately 50 job postings; approximately 17,000 practitioners; an 800 per cent posting surge; a projected 2,100 per cent demand increase	Four different producers, four different definitions, no shared sampling frame. Two originate with firms selling recruitment services	Mutually incompatible. Report the incoherence; do not select among them

The consequence is direct. There is no peer-reviewed estimate of an enterprise artificial intelligence project failure rate. Research motivated by such a rate is motivated by a

number that does not exist. We therefore motivate this work from documented mechanisms rather than from aggregate failure statistics, and we recommend that convention to others working in this area. Where a magnitude is genuinely required, McElheran et al. (2024) provide a defensible adoption baseline and Brynjolfsson et al. (2021) provide the theoretical account of why returns to a general-purpose technology lag investment while complementary intangible assets accumulate.

4.2. Definition

Synthesizing across the corpus and extending Kim and Hwang (2026), we define forward deployed engineering as follows.

Forward deployed engineering is the practice in which a supplier organization places engineering staff, who remain employees of the supplier and retain influence over its product, inside a customer organization for an extended period, in order to specify, integrate, adapt and operate a system within the customer's technical and organizational environment, and to return the resulting knowledge to the supplier's product.

Four elements do the work in this definition, and each excludes something. Retained employment excludes independent consulting and contract staffing. Retained product influence excludes the solutions architect and the professional services engineer, whose relationship to the roadmap is consumptive rather than generative. Extended presence inside the customer's environment excludes conventional pre-sales and post-sales support. The return path to the product excludes bespoke development performed for a single client without reuse. Remove any one element and the arrangement collapses into an existing and already-named category.

4.3. Distinction from adjacent disciplines

Forward deployed engineering is routinely conflated with six adjacent practices. The conflation is not merely terminological, because each adjacent practice carries assumptions about ownership and control that do not hold here. Table 2 sets out the comparison along the dimensions that the systems engineering analysis in Section 5 will use.

Table 2. Forward deployed engineering compared with adjacent practices.

Dimension	Adjacent practices	Forward deployed engineering
Unit of analysis	DevOps, MLOps, LLMOps and platform engineering all take a team inside a single firm as the unit	The vendor-client dyad, under divided managerial control
Ownership of the deployment target	Assumed by the practicing organization	Held by a different organization with independent authority
Requirements source	Internal stakeholders, reachable and accountable to the same management	External stakeholders, reachable only through an agreement, with their own priorities
Relationship to the product roadmap	Solutions engineering and professional services consume the roadmap	Generates roadmap input; the feedback path is constitutive, not incidental

Dimension	Adjacent practices	Forward deployed engineering
Persistence of the relationship	Consulting engagements terminate at handover	Continues through operation, because the system requires ongoing adaptation
Governing process assumption	AI engineering assumes the engineering organization controls the life cycle	Life cycle authority is split and negotiated at every phase boundary

4.4. Activity clusters and the process mapping

Coding the reported activity against ISO/IEC/IEEE 15288 produced two results. Most of the activity maps cleanly onto the standard's technical and technical management process groups. A minority does not map at all, and that minority is concentrated and coherent rather than residual. Table 3 reports the activity clusters we derived, following the taxonomy development procedure of Nickerson et al. (2013), with the process group each maps to and the party that the corpus indicates performs it.

Table 3. Activity clusters, process mapping and responsibility.

Activity cluster	ISO/IEC/IEEE 15288 process group	Responsibility
Problem discovery inside the client	Stakeholder needs and requirements definition	Joint, vendor-led
Data and systems integration	Integration; system requirements definition	Joint, contested
Customer-specific construction on the product	Implementation; design definition	Vendor
Transition into the client environment	Transition; operation	Joint
Sustained adaptation after go-live	Maintenance; validation	Joint, drifting to client over time
Evidence production for acceptance	Verification; validation; measurement	Unresolved. See Section 5.4
Responsibility and scope negotiation	Agreement processes: acquisition and supply	Joint, and continuously renegotiated
Client enablement and capability transfer	Knowledge management	Vendor to client
Commercial and pre-contract technical work	No mapping	Vendor
Roadmap feedback and productization	No mapping	Vendor
Relationship and expectation management	No mapping	Vendor

Three clusters have no home in the standard. Commercial and pre-contract technical work, roadmap feedback, and relationship management are not systems engineering processes as ISO/IEC/IEEE 15288 conceives them, yet the corpus describes them as central rather than peripheral to the role. This is the first substantive result of the mapping. A process standard designed for an organization building a

system for itself, or procuring one under a contract, has no representation for engineering work whose output is partly a system and partly an improvement to a commercial product line.

Two further observations follow from the responsibility column. Almost every cluster is joint. And the joint clusters are not jointly executed in a settled way: the corpus repeatedly describes the boundary being renegotiated as the engagement proceeds, which is why the agreement process group, normally a contractual formality executed once, becomes an operational process executed continuously.

4.5. Three structural properties

Beyond the mapping, three properties recur across the corpus with sufficient consistency and sufficient support from Tier 0 sources to be treated as structural rather than incidental.

The first is requirements unknowability. Multiple independent lines of evidence indicate that the requirements for a deployed artificial intelligence system cannot be fully specified before operation. Shankar et al. (2024) name criteria drift, the condition in which practitioners require evaluation criteria to grade system outputs but discover those criteria only by grading outputs. Barnett et al. (2024) conclude from three case studies that validation of a retrieval-augmented system is feasible only during operation and that robustness evolves rather than being designed in at the outset. Beede et al. (2020) demonstrate the same phenomenon environmentally: the operating conditions that determined system performance were not visible until the system was in the clinics. This is not a project management failure. It is a property of the class of system.

The second is the verification gap. Apostolou et al. (2026) report from sixteen practitioners across twelve companies that several organizations possessed demonstrated capability that they could not move into production, not because the capability was inadequate but because they had no mechanism for verifying outputs to a standard their processes would accept. They name this the capability-deployment verification gap. The corpus indicates that this gap falls precisely on the vendor-client boundary: the supplier possesses the system knowledge required to construct verification evidence, and the customer possesses the domain knowledge and the authority required to accept it.

The third is cross-boundary mirroring. Conway (1968) proposed and MacCormack et al. (2012) demonstrated that organizational communication structure is reproduced in system architecture. The forward deployed arrangement deliberately alters the communication structure across a firm boundary by placing supplier engineers inside the customer. If the mirroring hypothesis holds across that boundary, the composition and placement of the deployed team is an architectural decision about the customer's resulting system, not merely a staffing decision. The corpus does not test this, and we flag it as a proposition rather than a finding.

5. A systems engineering account

5.1. The engagement as a system of systems

The analytical move this paper proposes is to stop treating the deployed artificial intelligence application as the system under development, and to treat the vendor-client engagement as the system, with the application as one of its elements. Maier (1998) supplies the warrant. His criteria for a system of systems turn on operational independence, each constituent system being useful in its own right, and managerial independence, each being separately acquired and administered. A vendor and an enterprise customer satisfy both conditions unambiguously. Neither can direct the other. Each continues to operate when the engagement ends.

Treating the engagement as a system of systems has immediate consequences. Authority over the life cycle is distributed rather than held. Interfaces between constituent systems are governed by agreement rather than by design authority. Emergent behavior, including failure, arises from the interaction of independently managed parts. These are the standard concerns of systems-of-systems engineering, and they describe the difficulties reported in the corpus more accurately than any framing available in the software engineering or MLOps literatures, which assume unitary control.

5.2. A cross-organizational deployment life cycle

From the activity clusters and the responsibility mapping we derive the life cycle in Figure 1. It differs from conventional deployment life cycles in three respects. It is explicitly two-sided, showing where authority sits at each stage. It places evidence production, rather than release, at the transition boundary. And it closes two loops rather than one: an adaptation loop returning to the client environment, and a productization loop returning to the vendor's product, the second of which is what distinguishes forward deployed engineering from consulting.

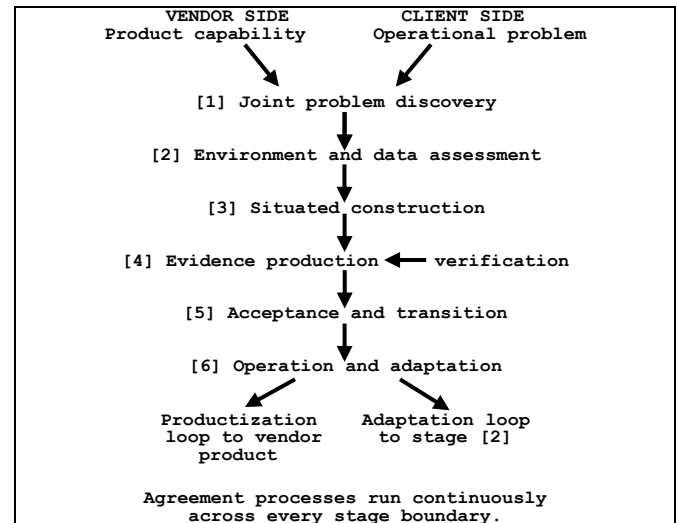


Figure 1. The cross-organizational deployment life cycle. Authority is divided at every stage; the agreement process group operates continuously rather than once.

Stage one is problem discovery conducted inside the client organization rather than elicited from it, which is the activity Tropos formalized as early requirements analysis (Bresciani et

al., 2004) and which conventional requirements engineering for machine learning does not model (Vogelsang and Borg, 2019). Stage two assesses the technical and organizational environment, and is where data cascades originate when it is performed inadequately (Sambasivan et al., 2021). Stage three is construction against the vendor's product within the client's constraints. Stage four, evidence production, is examined below. Stage five is acceptance and transition, the point at which authority formally passes. Stage six is operation with continuing adaptation, and the corpus is consistent that this stage does not terminate.

The two loops carry different content. The adaptation loop returns to stage two because the environment, not the requirement, is what has typically changed. The productization loop returns generalizable capability to the vendor's product. Where the productization loop is absent, the arrangement is consulting under a different name, and the vendor accumulates delivery cost without accumulating product value.

5.3. Where the standard breaks

ISO/IEC/IEEE 15288 accommodates most of this. Three points of breakage are worth stating precisely, because they are where a domain-specific contribution is required rather than a mere instantiation of the standard.

First, the standard's technical processes assume that requirements definition precedes verification. Under requirements unknowability this ordering cannot hold, and the V-model relationship between specification and verification, whose applicability to machine learning enabled systems Wu (2024) examines directly, loses its anchor. What is required is not a rearrangement of the V but an account of verification against criteria that are themselves outputs of the process.

Second, the standard treats agreement processes as bounding the life cycle rather than as running through it. In a forward deployed engagement the scope, the responsibility split and the acceptance criteria are renegotiated repeatedly, which makes acquisition and supply operational processes with a tempo comparable to the technical processes.

Third, the standard has no representation for the productization loop. A supplier that improves its product as a byproduct of a customer engagement is producing an output the standard does not model, and one that is often the economic justification for the arrangement.

5.4. Locating the verification gap

The verification gap identified by Apostolou et al. (2026) can now be placed precisely. It sits at stage four of Figure 1, between construction and acceptance, and it is a systems-of-systems problem rather than a testing problem. Verification evidence for a non-deterministic system operating on customer data can only be generated inside the customer environment, which the supplier does not control. It can only be judged adequate by customer stakeholders, who typically lack the technical basis to specify what adequate means for a probabilistic system. And it must satisfy an acceptance standard that, under requirements unknowability, was not available when the engagement was scoped.

The surrounding literature confirms that no instrument currently fills this position. The assurance case methodology developed for machine learning in autonomous systems targets supervised components in safety-critical settings rather than enterprise generative applications (Hawkins et al., 2021). Safety case work on frontier models addresses catastrophic risk (Clymer et al., 2024). The NIST AI Risk Management Framework specifies a Measure function without specifying measurement methods. Thelisson and Verma (2024) observe that the European regulation mandating conformity assessment provides limited guidance on conducting one. Practitioner evidence suggests the gap is currently closed by human inspection: Pan et al. (2025), surveying 306 practitioners running agentic systems in production, report that most systems execute a small number of steps before human intervention and that a large majority rely primarily on human evaluation.

We therefore identify the design of a verification and acceptance methodology for non-deterministic systems deployed into environments the supplier does not control as the highest-value open problem in this area, and note that it is simultaneously a research problem and a regulatory one.

5.5. Consistency with documented deployments

The account above was derived from a corpus about forward deployed engineering. As a preliminary consistency check we examined published deployment case studies written for other purposes. Beede et al. (2020) describe environmental conditions invalidating laboratory performance, which stage two predicts and which the reported deployment did not perform adequately. Sendak et al. (2020) describe an integration in which the documented work is overwhelmingly stakeholder alignment, role definition and training rather than modeling, and state that no playbook exists; the accompanying ethnography (Elish and Watkins, 2020) names the invisible repair work that made the system function, which corresponds closely to stages five and six. Granlund et al. (2021) report data ownership and responsibility division as the dominant multi-organization difficulties, corresponding to the continuous agreement processes in Figure 1. Barnett et al. (2024) report that validation was possible only in operation, corresponding to stage four.

We present this as consistency, not validation. The cases were not written about forward deployed engineering, the pool is small, and none describes a supplier engineer with retained product influence.

6. Discussion

6.1. Implications for research

The most immediate implication is that the deployment literature and the enterprise implementation literature need to be brought into contact. Each holds half of this problem. Software engineering for artificial intelligence understands the technical difficulty of deployment and assumes unitary organizational control; enterprise implementation research understands divided control and cross-boundary knowledge transfer but predates the technology by two decades and models

a third-party consultant rather than a supplier employee. Neither cites the other with any regularity.

We identify six research questions, ordered by our assessment of value. First, how should life cycle process models be reformulated when requirements are discoverable only through operation? This is a theory question whose answer would generalize well beyond this setting. Second, what do forward deployed engineers do, and how does that distribute across systems engineering process categories? This paper answers that from documents; it needs answering from practice. Third, where does the responsibility boundary fall across the life cycle, how is it negotiated, and what governs it? Fourth, what constitutes adequate verification evidence for a non-deterministic system in a customer environment, and can it be produced at acceptable cost? Fifth, does the mirroring hypothesis hold across the vendor-client boundary, such that deployed team structure predicts delivered architecture? MacCormack et al. (2012) supply an adaptable measurement approach. Sixth, and framing the rest, is forward deployed engineering durable or transitional?

Three studies would move this area quickly: an interview study of practitioners on both sides of the boundary; a systematic analysis of recruitment advertisements, which would yield a competency model and an activity distribution from countable public data at low cost; and a multi-case study selected for the phenomenon, which would permit genuine evaluation of the life cycle proposed here.

6.2. Implications for practice

Three implications follow for organizations operating this model. The first concerns the productization loop. Because it distinguishes the arrangement from consulting, and because it is the element most easily sacrificed under delivery pressure, it requires explicit measurement. An organization that cannot say what proportion of its product capability originated in customer engagements is not operating a forward deployed model; it is operating a professional services business with an unusual job title. Rasmussen (1997) supplies the mechanism: under competitive and efficiency pressure, work migrates toward the boundary of acceptable performance, and the discretionary loop is cut first.

The second concerns verification. Since evidence production sits on the organizational boundary and no established instrument exists, acceptance criteria and the division of evidentiary responsibility should be treated as first-class engineering and contractual artefacts, negotiated early and revised explicitly rather than left to emerge at go-live. The third concerns capability transfer: if the arrangement is transitional, client enablement is not a courtesy but the mechanism by which the engagement reaches a stable end state, and measuring it is measuring progress.

6.3. A transitional organizational form

We close with an argument about what forward deployed engineering is. The prevailing framing, in trade press and in recruitment, treats it as an emerging profession. We think that framing is likely to be wrong.

Cohen and Levinthal (1990) established that a firm's ability to absorb external knowledge depends on prior related knowledge, and Zahra and George (2002) distinguished potential absorptive capacity from the realized capacity to transform and exploit. Brynjolfsson et al. (2021) demonstrated that returns to a general-purpose technology lag investment while complementary intangible assets accumulate inside adopting organizations, producing a characteristic productivity J-curve. Read together, these accounts describe precisely the conditions under which an organization imports an external capability to compensate for one it does not yet possess, and the conditions under which that import ceases to be necessary.

Forward deployed engineering fits the pattern. It exists because customer organizations currently lack the complementary assets, in data infrastructure, in evaluation practice, in workflow redesign, required to convert a purchased model capability into operational value. Suppliers supply the missing capability directly, in the form of people. As client absorptive capacity develops, the arrangement should attenuate, first shifting toward enablement, then toward conventional product support. Practitioner accounts already anticipate obsolescence within a decade, and our reading is that they are describing a mechanism rather than a fashion.

The consequence for research is that the interesting question is not what the role is but what it indicates. A high density of forward deployed engineering in a sector is a measurement of the gap between what a technology can do and what its adopters can yet absorb. If that is right, the practice is a diagnostic instrument for technology diffusion, its own dissolution would be the measure of its success, and a research programme built around it should be designed to outlast it.

7. Conclusion

Forward deployed engineering has grown into a substantial industrial practice without attracting scholarly attention. This paper reports a structured multivocal review of 133 sources establishing that the peer-reviewed literature on the practice is empty, that the quantitative claims circulating about it do not survive provenance checking, and that the two literatures capable of explaining it have not been brought into contact.

We have argued that the practice is best analyzed by treating the vendor-client engagement, rather than the deployed application, as the system under development, and that doing so identifies a system of systems under divided managerial control. Mapping documented activity onto ISO/IEC/IEEE 15288 shows that most of the work is recognizable systems engineering performed across a firm boundary, that three activity clusters have no representation in the standard, and that the standard breaks at three points: the ordering of specification and verification, the tempo of agreement processes, and the absence of any representation for the productization loop. From this we derived a cross-organizational deployment life cycle and located the capability-deployment verification gap within it.

The principal limitation is that the account is documentary. Its central claims are testable and we have specified how. If the argument in Section 6.3 is correct, the most valuable work is

not the codification of a new profession but the study of what its existence reveals about the distance between technological capability and organizational absorption, and the design of the verification instruments that would let that distance be closed responsibly.

8. References

- Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., & Zimmermann, T. (2019). Software engineering for machine learning: A case study. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice* (pp. 291-300). IEEE/ACM.
- Amou Najafabadi, F., Bogner, J., Gerostathopoulos, I., & Lago, P. (2024). An analysis of MLOps architectures: A systematic mapping study. In *Software Architecture (ECSA 2024)*, LNCS 14889. Springer.
- Ancona, D. G., & Caldwell, D. F. (1992). Bridging the boundary: External activity and performance in organizational teams. *Administrative Science Quarterly*, 37(4), 634-665.
- Apostolou, S. A., Bosch, J., & Olsson, H. H. (2026). Agentic AI in industry: Adoption level and deployment barriers (arXiv:2605.14675). arXiv.
- Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., & Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering (CAIN 2024)*. ACM.
- Baxter, G., & Sommerville, I. (2011). Socio-technical systems: From design methods to systems engineering. *Interacting with Computers*, 23(1), 4-17.
- Beede, E., Baylor, E., Hersch, F., Iurchenko, A., Wilcox, L., Ruamviboonsuk, P., & Vardoulakis, L. M. (2020). A human-centered evaluation of a deep learning system deployed in clinics for the detection of diabetic retinopathy. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems*. ACM.
- Bogner, J., Verdecchia, R., & Gerostathopoulos, I. (2021). Characterizing technical debt and antipatterns in AI-based systems: A systematic mapping study. In *Proceedings of the IEEE/ACM International Conference on Technical Debt (TechDebt 2021)*, pp. 64-73. IEEE.
- Bresciani, P., Perini, A., Giorgini, P., Giunchiglia, F., & Mylopoulos, J. (2004). Tropos: An agent-oriented software development methodology. *Autonomous Agents and Multi-Agent Systems*, 8(3), 203-236.
- Brynjolfsson, E., Rock, D., & Syverson, C. (2021). The productivity J-curve: How intangibles complement general purpose technologies. *American Economic Journal: Macroeconomics*, 13(1), 333-372.
- Clymer, J., Gabrieli, N., Krueger, D., & Larsen, T. (2024). Safety cases: How to justify the safety of advanced AI systems (arXiv:2403.10462). arXiv.
- Cohen, W. M., & Levinthal, D. A. (1990). Absorptive capacity: A new perspective on learning and innovation. *Administrative Science Quarterly*, 35(1), 128-152.
- Conway, M. E. (1968, April). How do committees invent? *Datamation*.
- Elish, M. C., & Watkins, E. A. (2020). *Repairing innovation: A study of integrating AI in clinical care*. Data and Society Research Institute.
- Garousi, V., Felderer, M., & Mantyla, M. V. (2019). Guidelines for including grey literature and conducting multivocal literature reviews in software engineering. *Information and Software Technology*.
- Granlund, T., Kopponen, A., Stirbu, V., Myllyaho, L., & Mikkonen, T. (2021). MLOps challenges in multi-organization setup: Experiences from two real-world cases. In *Proceedings of the IEEE/ACM 1st Workshop on AI Engineering (WAIN)*. IEEE.
- Gregor, S., & Hevner, A. R. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37(2), 337-355.
- Hawkins, R., Paterson, C., Picardi, C., Jia, Y., Calinescu, R., & Habli, I. (2021). Guidance on the assurance of machine learning in autonomous systems (AMLAS) (arXiv:2102.01564). arXiv.
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75-105.
- International Organization for Standardization. (2023). *ISO/IEC/IEEE 15288:2023 Systems and software engineering: System life cycle processes* (2nd ed.). ISO.
- John, M. M., Olsson, H. H., & Bosch, J. (2021). Towards MLOps: A framework and maturity model. In *Proceedings of the 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. IEEE.
- Kim, J., & Hwang, H. (2026). Forward deployed engineering: A taxonomy and definition (SSRN Working Paper No. 6374660). <https://doi.org/10.2139/ssrn.6374660>
- Ko, D.-G., Kirsch, L. J., & King, W. R. (2005). Antecedents of knowledge transfer from consultants to clients in enterprise system implementations. *MIS Quarterly*, 29(1), 59-85.
- Kreuzberger, D., Kuhl, N., & Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11, 31866-31879.
- Liu, Y., Lo, S. K., Lu, Q., Zhu, L., Zhao, D., Xu, X., Harrer, S., & Whittle, J. (2024). Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents. *Journal of Systems and Software*, 220, 112278.
- MacCormack, A., Baldwin, C. Y., & Rusnak, J. (2012). Exploring the duality between product and organizational architectures: A test of the mirroring hypothesis. *Research Policy*, 41(8), 1309-1324.
- Maier, M. W. (1998). Architecting principles for systems-of-systems. *Systems Engineering*, 1(4), 267-284.
- Martinez-Fernandez, S., Bogner, J., Franch, X., Oriol, M., Siebert, J., Trendowicz, A., Vollmer, A. M., & Wagner, S. (2022). Software engineering for AI-based systems: A survey. *ACM Transactions on Software Engineering and Methodology*, 31(2), Article 37e.
- McElheran, K., Li, J. F., Brynjolfsson, E., Kroff, Z., Dinerso, E., Foster, L., & Zolas, N. (2024). AI adoption in America: Who, what, and where. *Journal of Economics and Management Strategy*, 33(2), 375-415.
- Nahar, N., Zhou, S., Lewis, G., & Kastner, C. (2022). Collaboration challenges in building ML-enabled systems: Communication, documentation, engineering, and process. In *Proceedings of the 44th International Conference on Software Engineering* (pp. 413-425). ACM.
- Nickerson, R. C., Varshney, U., & Muntermann, J. (2013). A method for taxonomy development and its application in information systems. *European Journal of Information Systems*, 22(3), 336-359.
- Paleyes, A., Urma, R.-G., & Lawrence, N. D. (2022). Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6), Article 114.
- Pan, M. Z., Arabzadeh, N., Cogo, R., Zhu, Y., Xiong, A., Agrawal, L. A., Mao, H., Shen, E., Patel, L., Liu, S., Shi, T., Liu, X., Davis, J. Q., Kang, D., Gonzalez, J. E., Sen, K., Song, D., Stoica, I., & Zaharia, M. (2025). Measuring agents in production (arXiv:2512.04123). arXiv.
- Rasmussen, J. (1997). Risk management in a dynamic society: A modelling problem. *Safety Science*, 27(2-3), 183-213.
- Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., & Aroyo, L. (2021). Everyone wants to do the model work, not the data work: Data cascades in high-stakes AI. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems* 28.
- Sendak, M., Ratliff, W., Sarro, D., Alderton, E., Futoma, J., Gao, M., ... O'Brien, C. (2020). Real-world integration of a sepsis deep learning technology into routine clinical care: Implementation study. *JMIR Medical Informatics*, 8(7), e15182.
- Shankar, S., Garcia, R., Hellerstein, J. M., & Parameswaran, A. G. (2024). We have no idea how models will behave in production until production: How engineers operationalize machine learning. *Proceedings of the ACM on Human-Computer Interaction*, 8(CSCW1).
- Steidl, M., Felderer, M., & Ramler, R. (2023). The pipeline for the continuous development of artificial intelligence models: Current state of research and practice. *Journal of Systems and Software*, 199, 111615.
- Thelissen, E., & Verma, H. (2024). Conformity assessment under the EU AI act general approach. *AI and Ethics*, 4, 113-121.
- Tuli, K. R., Kohli, A. K., & Bharadwaj, S. G. (2007). Rethinking customer solutions: From product bundles to relational processes. *Journal of Marketing*, 71(3), 1-17.
- Tushman, M. L. (1977). Special boundary roles in the innovation process. *Administrative Science Quarterly*, 22(4), 587-605.
- van Beijsterveld, J., & van Groenendaal, W. (2016). Solving misfits in ERP implementations by SMEs. *Information Systems Journal*, 26(4), 369-393.

- Vogelsang, A., & Borg, M. (2019). Requirements engineering for machine learning: Perspectives from data scientists. In Proceedings of the 6th International Workshop on Artificial Intelligence for Requirements Engineering (AIRE). IEEE.
- Wooldridge, M., Jennings, N. R., & Kinny, D. (2000). The Gaia methodology for agent-oriented analysis and design. *Autonomous Agents and Multi-Agent Systems*, 3(3), 285-312.
- Wu, J. (2024). An exploratory study of V-model in building ML-enabled software: A systems engineering perspective. In Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering (CAIN 2024). ACM.
- Xia, B., Lu, Q., Zhu, L., Xing, Z., Zhao, D., & Zhang, H. (2024). Evaluation-driven development and operations of LLM agents: A process model and reference architecture (arXiv:2411.13768). arXiv.
- Zahra, S. A., & George, G. (2002). Absorptive capacity: A review, reconceptualization, and extension. *Academy of Management Review*, 27(2), 185-203.